Classification Protocols with Minimal Disclosure*

Jinshuo Dong
Northwestern University
Evanston, IL, USA
jinshuo@northwestern.edu

Jason Hartline
Northwestern University
Evanston, IL, USA
hartline@northwestern.edu

Aravindan Vijayaraghavan
Northwestern University
Evanston, IL, USA
aravindv@northwestern.edu

ABSTRACT

We consider multi-party protocols for classification that are motivated by applications such as e-discovery in court proceedings. We identify a protocol that guarantees that the requesting party receives all responsive documents and the sending party discloses the minimal amount of non-responsive documents necessary to prove that all responsive documents have been received. This protocol can be embedded in a machine learning framework that enables automated labeling of points and the resulting multi-party protocol is equivalent to the standard one-party classification problem (if the one-party classification problem satisfies a natural independence-of-irrelevant-alternatives property). Our formal guarantees focus on the case where there is a linear classifier that correctly partitions the documents.

CCS CONCEPTS

• **Theory of computation → Theory and algorithms for application domains**; **Interactive proof systems**; • **Applied computing → Law**.

KEYWORDS

Classification; e-Discovery

ACM Reference Format:
Jinshuo Dong, Jason Hartline, and Aravindan Vijayaraghavan. 2022. Classification Protocols with Minimal Disclosure. In *Proceedings of the 2022 Symposium on Computer Science and Law (CSLAW '22), November 1–2, 2022, Washington, DC, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3511265.3550442

1 INTRODUCTION

This paper considers the multi-party classification problem that arises in document review for discovery in legal proceedings. The plaintiff (henceforth: Bob) issues a *request for production* to the defendant (henceforth: Alice). The legal team of Alice is then accountable for reviewing all documents and provides the responsive ones. Grossman and Cormack [7] show this manual process can be significantly improved by automation. A potential issue with the adoption of this technology, however, is that automation could reduce transparency and accountability, and the accuracy and completeness of this process relies critically on the accountability of Alice's legal team and its obligations under the rules of professional responsibility.

In addition to accountability, Gelbach and Kobayashi [4] identify significant problems with the above method for discovery. First, the defendant (Alice) bears most of the cost of reviewing and selecting the responsive documents and this asymmetry could lead the plaintiff (Bob) to exploit such costly requests. Second, it misaligns the incentives of the Alice's legal team and causes the team, on the grounds of professional responsibility, to conduct work to benefit its adversary.

Another possible way to implement requests for production places the effort and accountability on the plaintiff. Bob issues a request for production to Alice. Alice delivers all the documents to Bob's legal team. Bob's legal team identifies the responsive documents (and discards the non-responsive ones). Of course, there is now a risk that Bob's legal team might learn facts from the documents not specified in the request for production. Alice and Bob may enter into a confidentiality agreement under the order of the court to protect the disclosure of Alice's private information and Bob's legal team should operate under its obligations under the rules of professional responsibility.

This paper aims to understand multi-party binary classification protocols that rely as little as possible on external means of accountability. We aim for protocols that satisfy three main properties:

(1) (Correct) Bob receives all responsive documents.
(2) (Minimal) Alice minimizes privacy loss (as few non-responsive documents as possible are revealed to Bob).
(3) (Computationally Efficient): Algorithms run by all parties are computationally efficient.

We will also be interested in a fourth property which our protocol will satisfy:

(4) (Truthful) Alice's best strategy in the protocol is to truthfully reveal the set of relevant documents.

Our protocols will make use of a trusted third party, Trent. Trusted third parties are common in the design of protocols, and can often be replaced with secure multi-party computation (see Goldreich et al. [5], Yao [12]). For e-discovery of electronic mail, many companies already use third parties for email storage, and perhaps these third parties can take on the role of the trusted third party in our protocol. Alternatively, the court system could provide the trusted third party.

*This work began during the IDEAL Special Quarter on Data Science and Law organized by Jason Hartline and Dan Linna. Many thanks to Dan Linna for legal context and feedback on the project. The work was supported in part by NSF award CCF-1934931. The recommended version of the paper is available at https://arxiv.org/abs/2209.02690

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CSLAW '22, November 1–2, 2022, Washington, DC, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9234-1/22/11…$15.00
https://doi.org/10.1145/3511265.3550442

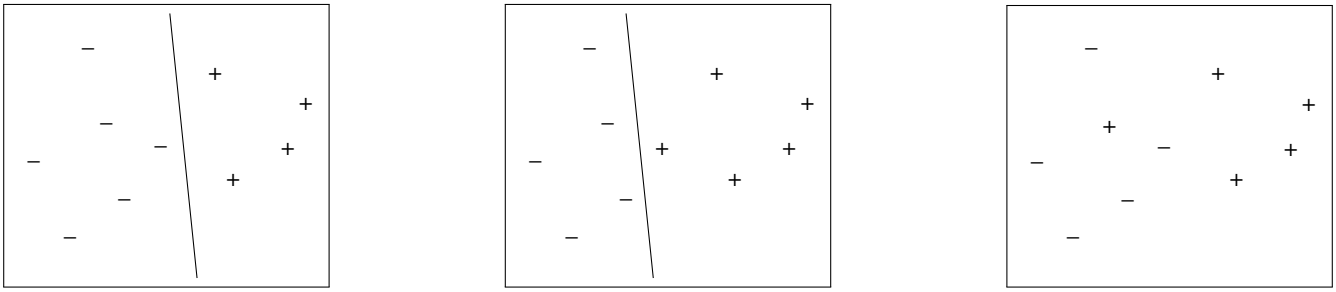

Figure 1: Left: The labeled points are linearly separable; Middle: The labeled points are linearly separable when the right-most negative point is relabeled as positive; Right: The labeled points are not linearly separable when the center negative point is relabeled as positive. The right-most negative point is critical; the center negative point is non-critical.

After introducing a trusted third party, there is now a third possible protocol: Bob communicates a classifier to Trent and Trent uses the classifier identify the responsive documents, checks these documents with Alice, and then communicates them to Bob. The problem with this approach is that, while we assume Bob's legal team can identify whether or not any given document is responsive, we do not assume that Bob can succinctly communicate such a labeling strategy in the form of a machine executable classifier. For Bob to produce such a classifier Bob needs real documents that only Alice possesses. We furthermore do not assume that Trent is capable of non-mechanical tasks.

Our protocol is based on two assumptions on the environment. First, we assume that Alice can and will provide Trent with all documents before the protocol begins. We view this assumption as much weaker than the current standard assumption of the leading paragraph where Alice is required to provide only the relevant documents. Providing all documents is a weaker requirement than providing the relevant documents because there is no potential discretion involved. Alice's legal team cannot claim to think an document was not responsive as a justification for not providing it. Our second assumption is that both Alice and Bob can determine responsiveness of documents and if there is a disagreement in responsiveness, that this disagreement can be resolved by the court.

Our problem is one of multi-party classification, dividing the documents (henceforth: points) into responsive (positive points) and non-responsive (negative points). We assume that there is a classifier that is consistent with the labeling of the documents (See Figure 1). A key construct in the protocol is, given a set of alleged positive points and a set of alleged negative points that are separable, identifying all the other points that could be labeled as positive by a consistent classifier. We will refer to these points as *leaked*. A key quantity for our protocol is the *critical points*: negative points that are leaked when all other points are known. It is easy to see that there is no way Trent can be convinced that a critical point is negative without confirming its negative label with Bob.

With these constructs we define the *critical points protocol*:

(0) Alice discloses all points to Trent.
(1) Alice discloses to Trent which of the points she alleges as positive.
(2) Trent assumes that all remaining points are negative and computes the alleged critical points.
(3) Trent sends the alleged positive and alleged critical points to Bob.
(4) Bob labels these points and sends the labels to Trent.
(5) Trent checks Bob's and Alice's labels agree (resolving any disagreement in court).
(6) Trent sends the leaked points corresponding to the correctly labeled points to Bob.

Our main protocol and results will be for binary classification with linear classifiers (See Figure 1). In these settings there is a projection from document space into a high-dimensional space of real numbers. Classifiers are given by hyperplanes that partition the space into two parts, the positives and the negatives. The assumption that there exists such a consistent classifier is known as the *realizable* setting or the *(linearly) separable* setting. Our main result is that for linear classification in the realizable setting, the critical points protocol is *correct*, *minimal*, and *computationally efficient*. (See Section 2 for formal definitions and theorem statements.)

While we focus on linear classification for exposition, our main result also extends to more powerful kernel-based classifiers like kernel support vector machines. Kernel methods embed the input space into a feature space that is higher-dimensional (potentially infinite dimensional), where the data is potentially linearly separable. Hence they capture more expressive hypothesis classes like polynomial threshold functions[1], and even neural networks in some settings (see for example Jacot et al. [8], Scholkopf and Smola [9], Shalev-Shwartz and Ben-David [11]). See Section 4 for details.

We can also show that the basic critical points protocol can be embedded within a machine learning framework that includes several of the technology-assisted review processes studied by Cormack and Grossman [2]. In this framework, there is a large

[1]The label of x is given by the sign of a polynomial $p(x)$; linear classifiers correspond to the special case of degree-1 polynomials.

universe of documents. This large universe of documents is sampled. The critical points protocol is run on the sample with labels provided by hand by the legal teams of Bob and Alice as specified. When the protocol terminates with Bob possessing both the critical negative points and the positive points, Bob selects a classifier that is consistent with these points. Bob reports this classifier to Trent who checks that it is consistent with the labeled points and then applies it to the universe of points and gives Bob all the points that are classified as positive. We prove that if Bob's classification algorithm satisfies *independence of irrelevant alternatives*, i.e., if the classifier selected is only a function of the set of consistent classifiers for the labeled points, then the outcome of this process is equivalent to the outcome of an analogous single-party classification procedure. In the case of linear classification and kernel-based classifiers, this can be instantiated with the support vector machine (SVM) algorithm that we prove satisfies the IIA property. (See Section 3 for formal statements.)

Related Work. Our work contributes to a growing literature on the theory of machine learning for social contexts. In this literature it is not enough for the algorithm to have good performance in terms of error, but it must also satisfy key definitions to be usable. Like a number of problems in this space, the gold-standard result is a reduction from the learning problem with societal concerns to the learning problem without such concerns. For example, Dwork et al. [3] construct fair classifiers from non-fair classifiers. A key perspective of this approach is it enables the machine learning algorithm designers to plug in their favorite algorithms, but results in a system with the desired societal properties, in their case, fairness. Our results for the machine learning framework in Section 3 are of a similar flavor: our protocol can be used in conjunction with any learning algorithm satisfying the IIA property to extend it to the multi-party setting.

Goldwasser et al. [6] consider interactive protocols for PAC (provably approximately correct) learning. They ask whether a verifier can be convinced that a classifier is approximately correct with far fewer labeled data points than it takes to identify a correct classifier. In the realizable case, the answer is yes. More generally, they show that there are classification problems where it is significantly cheaper in terms of labeled data points; and there are classification problems where it is no cheaper. Connecting to our model, their prover corresponds to Alice, their verifier corresponds to Bob. The big difference between their model and ours is that they assume that Bob (the verifier) can freely sample labeled data points. In our model Bob does not have access to the data without getting it from Alice. Moreover the main challenge in our setting is for Alice and Trent to convince Bob that no relevant documents were left out.

2 CRITICAL POINTS PROTOCOL: DEFINITIONS, PROTOCOL AND GUARANTEES

There are three parties: Alice (defendant), Bob (plaintiff) and Trent who is a trusted third party. Alice has a set of data points $S \subset \mathbb{R}^n$ (potentially the training samples), that is comprised of positive examples S_+ and negative examples S_- with their disjoint union denoted by $S_+ \sqcup S_- = S$. Alternately, each data point corresponds to a labeled example of the form (x, y) where $x \in \mathbb{R}^n$ and $y \in \{\pm 1\}$, where $y = +1$ if $x \in S_+$ and $y = -1$ if $x \in S_-$. These labeled examples are assumed to be (strictly) linearly separable i.e., there exists $d \in \mathbb{R}^n$ and $c \in \mathbb{R}$ such that $y = h(x) = \text{sign}(d \cdot x - c)$.

There are potentially several rounds of interaction between Alice, Bob and Trent. We will adhere to the revelation principle (Section 2.1) and restrict attention to protocols where Alice only interacts once and is asked to specify the positive labels S_+ (the other labels $S \setminus S_+$ are assumed to be the negative points S_-). For a truthful mechanism $\mathcal{M}$, we will denote by $\mathcal{M}(A_+, S_+) \subset S$ the set of points that are revealed to Bob eventually when Alice reports A_+ and the true labels are $S_+ \subset S$. Here $\mathcal{M}(A_+, S_+)$ is the output of the protocol. We aim for a protocol which satisfies the following four properties.

Definition 2.1. Protocol $\mathcal{M}$ properties on all data sets $S = S_+ \sqcup S_-$ and all reports $A_+ \subset S$:

(1) (Correct) The positive points are revealed to Bob, i.e., $\mathcal{M}(A_+, S_+) \supseteq S_+$.
(2) (Minimal) (If Alice reports truthfully) the protocol minimizes the number of negative points revealed, i.e., $|\mathcal{M}(S_+, S_+) \setminus S_+|$ is minimized.
(3) (Computational Efficiency) The algorithms run by all parties are computationally tractable.
(4) (Truthful) Alice's best strategy is to truthfully reveal the set of relevant documents, i.e., $A_+ = S_+$ minimizes $\mathcal{M}(A_+, S_+)$.

In the protocol we will define, Alice and Bob will be expected simply to label points. The complex computations will be mechanically performed by Trent, the trusted third party. The basic computation performed by Trent is the Leak operator which, given a subset of linearly separable points labeled as positives and negatives $A_+ \sqcup A_- = A$, determines the set of all the points S that are labeled as positive by some classifier consistent with the labels of A. Let $\mathcal{H} = \{h(x) = \text{sign}(d \cdot x - c) : d \in \mathbb{R}^n, c \in \mathbb{R}\}$ denote the set of all linear classifiers over $\mathbb{R}^n$.

Definition 2.2. The consistent classifiers for points $A_+ \sqcup A_- = A$ is $\mathcal{H}(A_+, A_-) =$

$$\{h \in \mathcal{H} : \forall x_+ \in A_+,\ h(x_+) = +1 \text{ and } \forall x_- \in A_-,\ h(x_-) = -1\}.$$

Definition 2.3. The leak operator $\text{Leak}(A_+, A_-)$ is all the points in S are classified by positive by some consistent classifier.

$$\text{Leak}(A_+, A_-) = \{x \in S : \exists h \in \mathcal{H}(A_+, A_-),\ h(x) = +1\} \cup A_-$$

Lemma 2.4. *For linear classification, whether or not a point $x \in S$ is in $\text{Leak}(A_+, A_-)$ is a linear classification problem (and can be computed in polynomial time).*

Proof. We first check if $x \in A_-$; if yes, $x \in \text{Leak}(A_+, A_-)$. If not, the goal is to check if there is a linear classifier $h \in \mathcal{H}$ that assigns a label +1 to all points in $A_+ \cup \{x\}$, and assigns the label −1 to all points in A_-. This is clearly a linear classification problem. This can be solved in polynomial time using the SVM algorithm (see Fact 3.4 in Section 3.2) or using a linear program. □

In the introduction the critical points were defined as the points that could are ambiguous with respect to the consistent classifiers when Alice reports positives as A_+ and all other points not labeled as positive by Alice are negatives. The definition of critical points requires that A_+ is linearly separable from $S \setminus A_+$.

Definition 2.5. The critical points for set $A_+ \subset S$ are

$$C^*(A_+) = \{x \in S : x \in \text{Leak}(A_+, S \setminus A_+ \setminus \{x\})\} \setminus A_+.$$

We refer to the following algorithm as the critical points protocol because, as we will subsequently prove (in Theorem 2.7),

$$\text{Leak}(A_+, C^*(A_+)) = A_+ \cup C^*(A_+). \tag{1}$$

Thus, if Alice truthfully reports $A_+ = S_+$ then the protocol terminates with the only negative points disclosed being $C^*(A_+)$.

Algorithm 1 Critical Points Protocol (CPP)

1: Alice sends all points S to Trent.
2: Alice sends alleged positive points $A_+ \subset S$ to Trent.
3: **if** A_+ and $S \setminus A_+$ are not separable **then**
4: Trent sends S to Bob and the protocol ends.
5: Trent computes critical points $C^*(A_+)$ and sends $A_+ \cup C^*(A_+)$ to Bob.
6: Bob labels the points and sends labels to Trent.
7: Trent checks that Bob and Alice's labels are consistent sending any disputed labels to be resolved by the court. Denote the resulting labeled points by $A'_+ \sqcup A'_-$
8: Trent sends $\text{Leak}(A'_+, A'_-)$ to Bob and the protocol ends.

Theorem 2.6. *When the data points S with positive samples S_+ and negative samples S_- are linearly separable, the critical points protocol (Algorithm 1) is (1) Correct, (2) Minimal i.e., for all A_+ $\mathcal{M}(A_+, S_+) \supseteq S_+ \cup C^*(S_+)$, and if Alice is truthful and reports S_+ to Trent, then $\mathcal{M}(S_+, S_+) = S_+ \cup C^*(S_+)$. Furthermore this protocol is (3) Computationally efficient and (4) Truthful.*

We remark that minimality of the CPP protocol holds in a stronger sense: in every *correct* protocol, Bob observes $S_+ \cup C^*(S_+)$; see Theorem 2.17 for a proof. Please also see Section 2.1 for a revelation principle.

We now proceed to the proof of Theorem 2.6. Note that Bob either sees all of S, or he sees $\text{Leak}(A'_+, A'_-)$ for some appropriate sets $A'_+ \subset S_+$ and $A'_- \subset S_-$.

Proof of Theorem 2.6. The proof of the theorem follows from the following three claims for any $A_+ \sqcup A_- \subset S$:

(1) $\text{Leak}(A_+, A_-)$ is non-increasing in A_+.
For fixed A_-, and consider $A'_+ \supset A_+$ that is separable from A_-. Separability of $A'_+ \sqcup A_-$ implies that the new points $A'_+ \setminus A_+$ were also previously leaked in $\text{Leak}(A_+, A_-)$. Moreover, for other points in S, there are now more constraints on separating hyperplanes so only fewer of them will be leaked. In total, no more points are leaked by $\text{Leak}(A'_+, A_-)$.

(2) $\text{Leak}(A_+, A_-)$ always contains $C^*(A_+)$.
Fix A_+, whether or not point x is leaked is monotone decreasing in $A_- \not\ni \mathbf{x}$. Adding points to A_- only adds constraints on separating hyperplanes making it only harder for x to be leaked. Since $x \in C^*(A_+)$ is leaked by $\text{Leak}(A_+, S \setminus A_+ \setminus \{x\})$ then by monotonicity x is leaked by $\text{Leak}(A_+, A_-)$ on all A_- separable from A_+.

(3) Nothing additional is leaked on $A_- = C^*(A_+)$,
i.e., $\text{Leak}(A_+, C^*(A_+)) = A_+ \cup C^*(A_+)$. This claim will be argued separately by Theorem 2.7, below.

These claims combine to give the theorem as follows. In the protocol, $A'_+ \subset S_+$ and $A'_- \subset S_-$. By the first claim we have $\text{Leak}(A'_+, A'_-) \supset \text{Leak}(S_+, A'_-)$. However, $\text{Leak}(S_+, A'_-) \supset S_- \cup A'_-$ and by the second claim $\text{Leak}(S_+, A'_-) \supset C^*(S_+)$;
thus, $\text{Leak}(S_+, A'_-) \supset S_- \cup A'_- \cup C^*(S_+)$ which, of course, is a superset of $S_+ \cup C^*(S_+)$ which is equal to $\text{Leak}(S_+, C^*(S_+))$ by the third claim. This latter minimal outcome is obtained by truthtelling. □

In the remainder of this section we prove equation (1) as Theorem 2.7.

Theorem 2.7. *For any $A_+ \subset S$, $\text{Leak}(A_+, C^*(A_+)) = A_+ \cup C^*(A_+)$.*

We have already argued that $\text{Leak}(A_+, C^*(A_+)) \supset A_+ \cup C^*(A_+)$, it suffices to show that no other points in $x \in S \setminus A_+ \setminus C^*(A_+)$ are in $\text{Leak}(A_+, C^*(A_+))$. To do so, we identify from A_+ a polyhedron S' that contains all of $S \setminus A_+$ and show that (a) its vertices V' are exactly $C^*(A_+)$ and (b) $\text{Leak}(A_+, V') = A_+ + V'$. Specifically, when the vertices of this polyhedron are disclosed, the other negative points, which are all within the polyhedron, are not leaked.

The polyhedron, $\text{Safe}(A_+, A_-)$ is defined as follows. Denote the maximum in direction $d \in \mathbb{R}^n$ in a set X by $\max_d(X) = \max_{x \in X} d \cdot x$ and, respectively, the minimum by $\min_d(X)$. Denote the (weakly) separating directions (See Figure 2) for linearly separable points A_+ and A_- by $\mathcal{D}(A_+, A_-)$ defined as

$$\mathcal{D}(A_+, A_-) = \{d \in \mathbb{R}^n : \max_d(A_-) \le \min_d(A_+)\}. \tag{2}$$

Define the convex subspace of points (a polyhedron) that would not be leaked by disclosing $A_+ \sqcup A_-$ as *safe points* (see Figure 2) and denote this subspace by

$$\text{Safe}(A_+, A_-) = \{x \in \mathbb{R}^n : \forall d \in \mathcal{D}(A_+, A_-), d \cdot x \le \max_d(A_-)\}. \tag{3}$$

I.e., a point is safe if in all separating directions there is a negative point disclosed in A_- that is at least as big (in this direction).

In what follows $\text{Verts}(X)$ denotes the vertices of convex set X; these are the points that are unique maximizers in any direction. The following lemma extends the fundamental theorem of linear programming to polyhedra like $\text{Safe}(A_+, A_-)$ that are defined by the separating hyperplanes between finite point sets. It shows that, if it is finite, the optimal point in some direction is attained at a vertex. Recall that the definition of separable for $A_+ \sqcup A_-$ is strict. This strictness is important. For example if all points in A_+ and A_- lie on the same hyperplane then A_+ is weakly on one side and A_- is weakly on the other, in a sense, they are weakly separated. In this case, $\text{Safe}(A_+, A_-)$ is a halfspace and halfspaces have no vertices. Moreover, if $S_+ \sqcup S_-$ are on the same hyperplane then there are no critical points $C^*(S_+) = \emptyset$. Thus, the strictness of separation will play an important role in the proof of the main theorem.

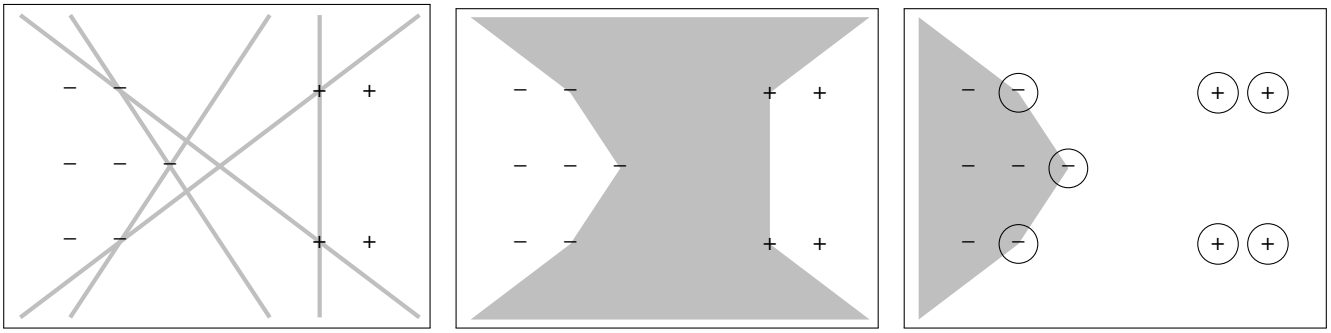

Figure 2: Left: Extreme separating hyperplanes are depicted. Center: all separating hyperplanes are depicted. Right: the space Safe(A_+, A_-) is depicted where $A_+ = S_+$ and $A_- = C^*(S_+)$ are the circled minuses. Note that Verts(Safe(A_+, A_-)) $= C^*(S_+)$. All circled points are disclosed by the protocol when Alice reports $A_+ = S_+$.

Lemma 2.8. *For separable $A_+ \sqcup A_-$ and $S' = Safe(A_+, A_-)$, if direction $d \in \mathbb{R}^n$ has a finite optimizer in S' then d is optimized in S' at a vertex $v \in Verts(S')$.*

Proof. The proof follows because Safe(A_+, A_-) does not contain a line (of infinite length) and the fundamental theorem of linear programming which states that for polyhedra that do not contain any line, every direction with a finite optimizer is optimized at a vertex (see Theorems 2.6 and 2.7 in Bertsimas and Tsitsiklis [1]).

Suppose the convex hulls of A_+ and A_- are strictly separated at distance $2\varepsilon > 0$ apart and consider a hyperplane h that separates them with distance ε from each. Because A_+ and A_- are both finite, their convex hulls are bounded and it is possible to rotate h small amount in any direction while still separating A_+ and A_-. A line intersects a hyperplane if and only if the line and hyperplane are not parallel. For any line, one of the small rotations of h is not parallel and, thus, is intersecting. This line, therefore, does not lie completely within Safe(A_+, A_-). □

We now observe that $\mathcal{D}$ and Safe behave the same way on second parameter given by any of A_-, $S' = \text{Safe}(A_+, A_-)$, and Verts$(S')$.

Lemma 2.9. *Let $S' = Safe(A_+, A_-)$, then*

- *$\mathcal{D}(A_+, A_-) = \mathcal{D}(A_+, S') = \mathcal{D}(A_+, Verts(S'))$, and*
- *$\max_d(A_-) = \max_d(S') = \max_d(Verts(S'))$ for all directions $d \in \mathcal{D}(A_+, A_-)$.*

Proof. For the first bullet: For the first equality, all points in A_- are safe so $S' \supset A_-$. $\mathcal{D}(A_+, \cdot)$ is bigger when its second argument is smaller. But every $d \in \mathcal{D}(A_+, A_-)$ is also in $\mathcal{D}(A_+, S')$ as the only points we add in S' are smaller than the largest point in A_+ in direction d. For the second equality, Lemma 2.8 implies $\mathcal{D}(A_+, S') = \mathcal{D}(A_+, \text{Verts}(S'))$.

For the second bullet: By the definition of safe, $\max_d(A_-) = \max_d(S')$. Specifically, all points we add are worse than points in A_- in all relevant directions, but points in A_- are also contained in S' so the maximum values over these sets in direction d direction are equal. Lemma 2.8 implies $\max_d(S') = \max_d(\text{Verts}(S'))$. □

Corollary 2.10. *Let $S' = Safe(A_+, A_-)$, then*

$$Safe(A_+, A_-) = Safe(A_+, S') = Safe(A_+, Verts(S')).$$

Proof. By Lemma 2.9 all the terms in the definition of Safe that depend on the second parameter are the same. □

We conclude that if the vertices of Safe are disclosed, then no other negative points are leaked.

Lemma 2.11. *For $S' = Safe(A_+, S \setminus A_+)$ and $V' = Verts(S')$, then*

$$\text{Leak}(A_+, V') = A_+ \cup V'.$$

Proof. $A_+ \cup V'$ are leaked by definition. By Corollary 2.10, Safe$(A_+, V') = \text{Safe}(A_+, S \setminus A_+)$. By the definition of Safe, no other points in Safe$(A_+, S \setminus A_+) \supset S \setminus A_+$ are leaked. □

With a view towards characterizing the vertices of Safe, the following lemma shows that all directions with finite maximizers within Safe are contained in $\mathcal{D}$. In what follows cone(X) denotes the set of points obtained by taking non-negative linear combinations of points in X. A set X is a convex cone if and only if cone$(X) = X$; it is said to be finitely generated if there exists a finite set of points $v_1, \ldots, v_m$ such that $X = \text{cone}(\{v_1, \ldots, v_m\})$.

Lemma 2.12. *For any A_+, A_-, $\mathcal{D}(A_+, A_-)$ is a convex cone. Moreover for any direction $d \in \mathbb{R}^n$, $\max_{x \in Safe(A_+, A_-)} d \cdot x$ is finite if and only if $d \in \mathcal{D}(A_+, A_-)$.*

Proof. We first observe that $\mathcal{D}(A_+, A_-)$ defined in (2) can equivalently be described as

$$\mathcal{D}(A_+, A_-) = \left\{d : \forall a_- \in A_-, a_+ \in A_+,\ d \cdot (a_- - a_+) \leqslant 0\right\}, \quad (4)$$

which corresponds to the solution set of a system of homogenous linear inequalities. Hence $\mathcal{D}(A_+, A_-)$ is a convex cone since it is closed under non-negative combinations.

We now prove the second part. One direction is easy: if $d \in \mathcal{D}(A_+, A_-)$, then by the definition of Safe, we have $\max_{x \in \text{Safe}(A_+, A_-)} \leqslant \max_d(A_-)$ which is bounded.

The other direction is more challenging and involves proving that the every direction with a finite maximizer over Safe(A_+, A_-) is in $\mathcal{D}(A_+, A_-)$. We would like to use linear programming (LP) duality to prove that every direction is in the cone $\mathcal{D}(A_+, A_-)$.

However, it is not clear that Safe is a polyhedron to apply LP duality i.e., described by a finite set of linear inequalities. Note that from (4), we see that $\mathcal{D}(A_+, A_-)$ is described by a finite number of constraints. Hence by Weyl's theorem on polyhedral cones (see Schrijver [10]), $\mathcal{D}(A_+, A_-)$ is also a finitely generated cone. However this does not suffice since the constraint for each direction is of the form $d \cdot x \leqslant \max_d(A_-)$. [2]

We first show that Safe can indeed be described by a finite number of linear inequalities, and then use LP duality to complete the argument. Let $\ell = |A_-|$ and $A_- = \{a_1, a_2, \ldots, a_\ell\}$. We define convex sets $\mathcal{D}_i$ and Safe_i (here we suppress the arguments A_+, A_- for easier notation) as follows :

$$\forall i \in [\ell],\ \mathcal{D}_i \coloneqq \{d \in \mathcal{D}(A_+, A_-) : d \cdot a_i = \max_d(A_-)\}. \quad (5)$$

$$\text{Safe}_i \coloneqq \left\{x : \forall d \in \mathcal{D}_i,\ d \cdot x \leqslant d \cdot a_i\right\} \quad (6)$$

$$\text{Then, Safe}(A_+, A_-) = \bigcap_{i \in [\ell]} \text{Safe}_i. \quad (7)$$

We now show that each of the convex sets Safe_i (and hence Safe) is polyhedral i.e., described by a finite set of constraints. For each $i \in [\ell]$, $\mathcal{D}_i$ is also a convex cone that is finitely generated. This is because $\mathcal{D}_i$ is described exactly by the finite set of linear constraints as $\mathcal{D}_i = \{d : \forall j \in [\ell],\ d \cdot (a_j - a_i) \leqslant 0\}$. Hence by the Weyl theorem for polyhedral cone duality (see Schrijver [10]), there exists finite $r_i \in \mathbb{N}$ such that the set of vectors $v_{i1}, \ldots, v_{ir_i}$ such that $\mathcal{D}_i = \text{cone}(v_{i1}, \ldots, v_{ir_i})$. Now we see that

$$\text{Safe}_i = \{x : \forall j \in [r_i],\ v_{ij} \cdot x \leqslant v_{ij} \cdot a_i\}. \quad (8)$$

The subset inclusion in (8) is obvious. The other direction just follows because $v_{i1}, \ldots, v_{ir_i}$ generate the cone. This shows that for each $i \in [\ell]$, Safe_i is polyhedral i.e., described by a finite set of linear constraints. Hence from (7) we have

$$\text{Safe}(A_+, A_-) = \{x : \forall i \in [\ell], \forall j \in [r_i], v_{ij} \cdot x \leqslant b_{ij}\}, \quad (9)$$

which is described by a finite number of constraints $r_{\text{tot}} \coloneqq \sum_{i=1}^{\ell} r_i$.

Finally, we now use linear programming duality to show that d has a finite maximum over Safe if and only if $d \in \mathcal{D}(A_+, A_-)$. Consider the linear program (LP) given by $\max_{x \in \mathbb{R}^n} d \cdot x$ such that x satisfies the constraints in (9). By LP duality, this LP has a finite maximum (i.e., bounded) if and only if its dual LP is feasible i.e., there exists a non-negative vector $y \geqslant 0$ in r_{tot} dimensions with such that

$$\sum_{i=1}^{\ell} \sum_{j=1}^{r_i} y_{ij} v_{ij} = c.$$

In other words, if $\max_d(\text{Safe}(A_+, A_-))$ is finite (bounded), then $d \in \text{cone}(\{v_{ij} : i \in [\ell], j \in [r_i]\}) \subset \mathcal{D}(A_+, A_-)$. □

Now we show that the vertices of Safe are equal to the critical points.

[2]For example even if $d = v_1 + v_2$, the RHS of the constraint $\max_d(A_-)$ could be smaller than $\max_{v_1}(A_-) + \max_{v_2}(A_-)$; hence some of the constraints that define Safe are not necessarily implied by constraints on just the generators of the cone $\mathcal{D}(A_+, A_-)$.

Lemma 2.13. *For any linearly separable set $S_+ \sqcup S_- = S$, the critical points are the vertices of the safe points, i.e.,*

$$C^*(S_+) = \textit{Verts}(\textit{Safe}(S_+, S_-)).$$

Proof. Let $S' = \text{Safe}(S_+, S_-)$ and $V' = \text{Verts}(S')$. The proof follows from the following two statements that we establish:

$$V' \subseteq S_- \quad (10)$$

$$V' \subseteq C^*(S_+). \quad (11)$$

$$V' \supseteq C^*(S_+). \quad (12)$$

Consider any vertex $x' \in S'$, and let $d' \in \mathbb{R}^n$ be the direction that it uniquely maximizes within S'. From Lemma 2.12 we have that $d' \in \mathcal{D}(S_+, S_-)$ since d' has a finite maximizer in $\text{Safe}(S_+, S_-)$. From the definition of $\text{Safe}(S_+, S_-)$, for every direction $d \in \mathcal{D}(S_+, S_-)$ (and in particular d'), there exists an element of S_- that achieves $\max_{x \in S'} d \cdot x$. Hence $x' \in S_-$ since x' is the unique maximizer in S'. This establishes (10).

We now show $V' \subseteq C^*(S_+)$. As before let $x' \in V'$ and d' be a direction that it uniquely maximizes within $S' \supseteq S_-$. Hence there is a linear classifier consistent with $S_+ \cup \{x'\}$ labeled positive, and $S_- \setminus \{x'\}$ labeled negative. Hence $x' \in C^*(S_+)$ as required for (11).

Finally to show (12), suppose $x' \in C^*(S_+)$. By definition, there is a linear classifier separating $S_+ \cup \{x'\}$ (as positives) and $S_- \setminus \{x'\}$ (as negatives). Moreover, S_+ and S_- are also separable (in particular it labels $x' \in S_-$ as a negative example). Hence by convexity, there exists a direction d' such that x' is the unique maximizer among the S_-, and $d' \cdot x' < \min_d(S_+)$. Hence $d' \in \mathcal{D}(S_+, S_-)$ and from (10), we have that x' is a unique maximizer in S' of d'. Hence (12). This concludes the proof. □

Proof of Theorem 2.7. By Lemma 2.13, the critical points of $C^*(S_+)$ are equal to the vertices of $\text{Safe}(S_+, S_-)$. Plugging this equivalence into Lemma 2.11, we have $\text{Leak}(S_+, C^*(S_+)) = S_+ \cup C^*(S_+)$. □

2.1 Truthful Protocols

In protocols for e-discovery Alice desires to (a) hide positive data points and (b) reduce the disclosure of negative data points. In a correct protocol, all positive data points are revealed, thus, Alice faces only the problem of reducing the discosure of negative data points. Following the standard framework from mechanism design in economics and computer science, we define the protocol properties of direct and truthful and provide a revelation principle. For this discussion we view Alice's interaction in the protocol. Note that all correct protocols disclose all of the positive points S_+; thus, minimizing the number of points disclosed in a correct protocol is equivalent to minimizing the number of negative points disclosed.

Definition 2.14. Given a known set of points S, a direct protocol $\mathcal{M} : 2^S \times 2^S \to 2^S$ maps the sets of alleged positives (of Alice) and true positives (as can be verified by Bob) to a set of disclosed points (to Bob).

Definition 2.15. A direct protocol is *truthful* if for all $S_+ \sqcup S_- = S$, Alice's optimal strategy is to truthfully report S_+.

Proposition 2.16 (Revelation Principle). *For any protocol $\mathcal{M}$ and optimal strategy σ of Alice mapping positive points to messages in the protocol (which minimizes the total number of data points disclosed), there is a truthful and direct protocol $\mathcal{M}^R$ with the same outcome under truthtelling (as under protocol $\mathcal{M}$ with strategy σ).*

Proof. Define the revelation protocol as $\mathcal{M}^R(A_+, S_+)$ as follows:

(1) Simulate strategy $\sigma(A_+)$ in $\mathcal{M}$ assuming A_+ are the true positives.
(2) Given the transcript of this simulation, attempt the same interaction as $\sigma(A_+)$ with the real true positives S_+.
(3) If the behavior of $\mathcal{M}$ with true positives S_+ is ever deviates from the simulated transcript, then reveal the full dataset S to Bob. (Otherwise, the outcome is identical to $\mathcal{M}(\sigma(A_+), A_+)$.)

We now argue that the optimal strategy in $\mathcal{M}^R$ is to report $A_+ = S_+$. Suppose some $A_+ \neq S_+$ gives a strictly better outcome. Note: it must be that the outcomes of $\mathcal{M}(\sigma(A_+), A_+)$ and $\mathcal{M}(\sigma(A_+), S_+)$ are the same, otherwise, the difference would be detected and the full set S would be disclosed to Bob. In this case, however, with true positives S_+ following $\sigma(A_+)$ rather than $\sigma(S_+)$ in $\mathcal{M}$ gives a strictly better outcome, which contradicts the optimality of σ for $\mathcal{M}$. □

2.2 Minimal Protocol

In this section we prove that every correct protocol discloses the critical points $C^*(S_+)$ on dataset $S_+ \sqcup S_- = S$; thus, the critical points protocol is optimal.

Theorem 2.17. *Every correct protocol $\mathcal{M}$ on dataset $S_+ \sqcup S_- = S$ discloses a set of points that contains $C^*(S_+)$.*

Proof. By Proposition 2.16, it is without loss to assume $\mathcal{M}$ is truthful. By the definition of truthful protocols, Alice cannot have fewer points disclosed by reporting non-truthfully. Suppose for a contradiction that a point $x^* \in C^*(S_+)$ is not disclosed in $S_+ \sqcup A_- = \mathcal{M}(S_+, S_+)$, i.e., $x^* \in S_-$ but $x^* \notin A_-$.

Recall $C^*(S_+) = \{x \in S : \text{Leak}(S_+, S \setminus S_+ \setminus \{x\})\}$ is the points that are each labeled as positive by some consistent classifier with respect to positives S_+ and negatives $S \setminus S_+ \setminus \{x\}$. Monotonicity of Leak implies that x^* is in $\text{Leak}(S_+, A_-)$ as $A_- \subseteq S \setminus S_+ \setminus \{x^*\}$. By the definition of Leak there is a consistent classifier that labels $S_+ \cup \{x^*\}$ as positive and A_- as negative.

Since x^* is in $\text{Leak}(S_+, A_-)$, there exists a separating hyperplane for $S_+ \cup X$ (with $X \ni x^*$) and the remaining points (which contains A_-). Thus, Lemma 2.18 (below) can be applied where $S_+ \cup X$ is separable but $x^* \in X$ is not disclosed on $\mathcal{M}(S_+, S_+ \cup X)$, a contradiction to the correctness of $\mathcal{M}$ as $x^* \in X$ is considered a positive point in the execution of $\mathcal{M}(S_+, S_+ \cup X)$ □

Lemma 2.18. *In any direct protocol $\mathcal{M}$, if $X \sqcup S_+$ is separable and $X \cap \mathcal{M}(A_+, S_+) = \emptyset$ (i.e., X is not disclosed by $\mathcal{M}$ with X are negative) then $\mathcal{M}(A_+, S_+) = \mathcal{M}(A_+, S_+ \cup X)$ (i.e., $\mathcal{M}$ on A_+ discloses the same points when points X are all positive or all negative).*

Proof. Separability of $X \cup S_+$ implies that $\mathcal{M}(A_+, S_+ \cup X)$ is well defined. By definition a protocol is only a function of its input, in this case, A_+ and the points that it discloses. Since X is not disclosed in $\mathcal{M}(A_+, S_+)$ then $\mathcal{M}(A_+, S_+ \cup X)$ has the same result, and X is not disclosed by it as well. □

3 MACHINE LEARNING GUARANTEES

In this section we consider the machine learning framework related to multi-party e-discovery (MPeD). We show how the protocol defined in the previous section (for linear classification), when instantiated with *any* training algorithm that satisfies a natural property, that we call the *independence of irrelevant alternatives (IIA)*, achieves the same learning guarantees as single-party e-discovery (SPeD). We provide a reduction from MPeD to SPeD; this shows that our protocol suffers no loss in generalization or sample complexity compared to the standard single-party setting. Finally, we instantiate this reduction using the classic support vector machine (SVM) algorithm, by showing that it satisfies the IIA property.

3.1 Machine learning framework

We start by recalling the single party e-discovery (SPeD) framework which corresponds to a standard machine learning pipeline, involving a training algorithm run on the training set S to find a good classifier $\hat{h}$, and then applying this classifier on the entire dataset U. $\text{Alg}_{\mathcal{H}}$ will denote a learning algorithm for the hypothesis class $\mathcal{H}$ that takes in labeled samples as input, and outputs a hypothesis in $\mathcal{H}$ consistent with the labeled samples.

Algorithm 2 ML framework for Single-Party e-Discovery (SPeD)

Input: Unlabeled dataset U and the hypothesis class $\mathcal{H}$.
Output: Data points with positive labels $U_+ \subseteq U$.

1: Sample the training data set $S \subseteq U$ (likely with $|S| \ll |U|$).
2: (Hand-)Label S to get S_+ and S_-.
3: Use $\text{Alg}_{\mathcal{H}}$ to learn classifier $\hat{h} \in \mathcal{H}$ on the labeled data with positives S_+ and negatives S_-.
4: Apply classifier $\hat{h}$ on U to get U_+

In the Multi-Party e-Discovery (MPeD) framework, there are three parties Alice, Bob and Trent that perform different functions. Alice first sends the entire dataset U to Trent. Trent generates the training samples $S \subseteq U$ and sends it to Alice. Alice, Trent and Bob engage in a protocol as in Section 2. At the end of the protocol, Bob is given a set of positive examples S'_+ (which is hopefully S_+) and some other negative samples S'_- (which is hopefully much smaller than S_-), which he then uses to train a classifier $\hat{h}$ that is used to classify the entire dataset U.

We want the classifier that is output in multi-party ML framework to be as good as the classifier in the single-party setting, irrespective of Alice's actions; ideally, it also maintains the same statistical properties (e.g., sample complexity) as the single-party setting. However, the choice of the learning algorithm is important, since the algorithm is trained on a different set of labeled samples ((S'_+, S'_-) as opposed to (S_+, S_-)). The following property of the

Algorithm 3 ML framework for Multi-Party e-Discovery (MPeD)

Input: Alice has unlabeled data points U. The hypothesis class is $\mathcal{H}$ is known publicly.

Goal: Bob receives data points with positive labels $U_+ \subseteq U$.

1: Alice sends entire dataset U to Trent.
2: Trentsamples the training data set $S \subseteq U$ (likely with $|S| \ll |U|$).
3: Trent sends S to Alice.
4: Alice, Trent, Bob participate in the *critical points protocol* (Algorithm 1) of Section 2. At the end of it, Bob receives labeled samples S'_+ and S'_-.
5: Bob uses $\text{Alg}_{\mathcal{H}}$ to learn a classifier $\hat{h} \in \mathcal{H}$ consistent with the labeled data S'_+ and S'_-, and sends $\hat{h}$ to Trent.
6: Trent checks the consistency of $\hat{h}$ with S'_+, S'_- and applies $\hat{h}$ on U to get U_+ and sends it to Bob.

learning algorithm will play a crucial role. Recall that $\mathcal{H}(S_+, S_-)$ denotes the set of hypothesis in $\mathcal{H}$ consistent with the labeled data given by positives S_+ and negatives S_-; also for a linearly separable data set $S_+ \sqcup S_-$, the critical points are denoted by $C^*(S_+)$.

Definition 3.1. (IIA property) A learning algorithm $\text{Alg}_{\mathcal{H}}$ is said to satisfy *independence of irrelevant alternatives* (IIA) if for any S_+ and $S_- \supseteq C^*(S_+)$ that is separable and for any $S'_- \supseteq C^*(S_+)$, we have that $\text{Alg}_{\mathcal{H}}(S_+, S_-) = \text{Alg}_{\mathcal{H}}(S_+, S'_-)$.

One can also define a potentially stronger notion of IIA where $\mathcal{H}(S_+, S_-) = \mathcal{H}(T_+, T_-) \implies \text{Alg}_{\mathcal{H}}(S_+, S_-) = \text{Alg}_{\mathcal{H}}(T_+, T_-)$, but the above weaker notion suffices for our purposes.

We focus on the setting where the data set U is linearly separable i.e., $\mathcal{H}$ is the set of linear classifiers, and there is an $h^* \in \mathcal{H}$ that is consistent with the true labels of U. We now show that we can use our protocol from Section 2 in Step 4 of the above framework, along with any algorithm that satisfies the IIA property to achieve the same statistical guarantees as the single-party ML setting. Note that the IIA property pertains only to the learning algorithm that is employed by Bob in Step 6 of the Algorithm 3.

Theorem 3.2. *Suppose the data set U is linearly separable and the learning algorithm $\text{Alg}_{\mathcal{H}}$ satisfies the IIA property. Then with the same sampling procedure (to produce S), the outputs of Algorithm 2 (SPeD) and Algorithm 3 (MPeD) are identical.*

We remark that there can be randomness in the sampling procedure and potential random choices in the learning algorithm $\text{Alg}_{\mathcal{H}}$; so U_+ is a random set. The guarantee of Theorem 3.2 is that the distributions of U_+ are the same. Alternatively, fixing the random choices in the sampling, and in the algorithm $\text{Alg}_{\mathcal{H}}$, the set U_+ is the same.

Proof of Theorem 3.2. The proof follows easily by combining the guarantees of Theorem 2.6 and the IIA property of $\text{Alg}_{\mathcal{H}}$. First, from Theorem 2.6, we know that irrespective of the actions of Alice, Bob receives S_+ and $C^*(S_+)$, where $C^*(S_+)$ denotes the critical points. Hence, from the IIA property of $\text{Alg}_{\mathcal{H}}$, the classifier $\hat{h}$ that is produced is identical (for the same random choices of the algorithm $\text{Alg}_{\mathcal{H}}$. Hence U_+ is identical in both cases. □

For a given linearly separable dataset there may be several potential linear classifiers consistent with it (and so too for (S_+, S_-)). Furthermore, not all algorithms for linear classification may satisfy the IIA property (e.g., the popular Perceptron algorithm does not satisfy IIA). However, the well-known SVM algorithm that finds the maximum margin classifier for the given linearly-separable dataset satisfies the IIA property (see Lemma 3.6 in the next section). Hence we can instantiate Theorem 3.2 for linear classifiers by using the SVM algorithm as follows. (The proof just follows by combining Theorem 3.2 and Lemma 3.6.)

Theorem 3.3. *Suppose the data set U is linearly separable and the learning algorithm $\text{Alg}_{\mathcal{H}}$ is the SVM algorithm given in Section 3.2. Then with the same sampling procedure (to produce S), the outputs of Algorithm 2 (SPeD) and Algorithm 3 (MPeD) are identical.*

This theorem shows that the multi-party e-discovery protocol given in Algorithm 3 incurs no loss compared to the single-party setting (Algorithm 2) in terms of properties of the output classifier $\hat{h}$. In particular, any statistical property (like test error or generalization guarantee) of the classifier $\hat{h}$ transfer over to the multi-party setting with no loss in the statistical efficiency. See Section 4 for the extension to kernel classifiers.

3.2 Support Vector Machines (SVM) and Properties

We now describe the support vector machine (SVM) algorithm which is used for learning linear classifiers for a given set of labeled samples in high-dimensional spaces. We will also prove the IIA property and see some facts about the SVM algorithm that will be useful in the next section.

The setting is as follows. We are given a set of labeled samples $T = \{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\} \subset \mathbb{R}^n \times \{\pm 1\}$. The goal is to find a linear classifier $(w, b) \in \mathbb{R}^n \times \mathbb{R}$ such that $\forall i \in [m],\ y_i(w \cdot x_i + b) > 0$ if such a classifier exists i.e., it is linearly separable.

The Hard-SVM algorithm finds the linear classifier that separates the positive and negative samples with the largest possible margin (if the data is linearly separable). For a classifier (w, b) with $\|w\|_2 = 1$, the margin of a sample (x, y) is the distance between a point x and the hyperplane (w, b) and is given by $\max\{y(w \cdot x + b), 0\}$. Note that a linear classifier does not change by scaling. The following claim shows that the Hard-SVM problem of finding maximum margin linear classifier can be reformulated in either of the following two ways.

Fact 3.4. *(See e.g., chapter 15 of Shalev-Shwartz and Ben-David [11]) Given a set of linearly separable labeled samples $\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\} \subset \mathbb{R}^n \times \{\pm 1\}$, consider the following optimization problems:*

$$w^*, b^* = \underset{(w,b):\|w\|_2=1}{\arg\max} \min_{i \in [m]} |w \cdot x_i + b| \quad \text{s.t. } \forall i \in [m],\ y_i(w \cdot x_i + b) > 0 \tag{13}$$

$$w^\dagger, b^\dagger = \underset{(w,b)}{\arg\min} \|w\|_2^2 \quad \text{s.t. } \forall i \in [m],\ y_i(w \cdot x_i + b) \geqslant 1. \tag{14}$$

The optimization problems (13) *and* (14) *are essentially equivalent, with the optimizers related as* $w^* = \frac{w^\dagger}{\|w^\dagger\|_2}, b^* = \frac{b^\dagger}{\|w^\dagger\|_2}$. *Moreover* (14) *is a convex program that can be solved in polynomial time.*

While (13) more directly captures the maximum margin formulation, (14) more clearly illustrates why it is a convex program that can be solved in polynomial time. When the data is not linearly separable, the linear constraints that define the convex program in (14) just become infeasible.

We have the following characterization that the solution of the Hard-SVM problem can be expressed as a linear combination of points which are all at the minimum distance (of exactly $1/\|w^\dagger\|_2$) from the separating hyperplane (these points are called the support vectors). Note that the optimal solution of (14) when it exists, is always unique (the objective is strongly convex).

Fact 3.5. *(Theorem 15.8 in Shalev-Shwartz and Ben-David [11]) Let* $w^\dagger, b^\dagger$ *denote the optimal solution of* (14)*, and let* $I = \{i \in [m] : y_i(w^\dagger \cdot x_i) = 1\}$. *Then there exists coefficients* $\alpha_1, \ldots, \alpha_m \in \mathbb{R}$ *such that* $w^\dagger = \sum_{i \in I} \alpha_i x_i$.

The above fact can be used to reformulate the objective (14) in terms of the unknowns $\alpha_1, \ldots, \alpha_m$ and the inner products between the points $\{x_i \cdot x_j : i, j \in [m]\}$ (as opposed to the points $\{x_i : i \in [m]\}$ themselves).

Lemma 3.6. *(SVM satisfies IIA property) The Hard-SVM procedure given by* (14) *is IIA (as per Definition 3.1) i.e., for any* S_+ *and* $S'_- \supseteq C^*(S_+)$ *(so that* $\mathcal{H}(S_+, S'_-) = \mathcal{H}(S_+, S_-)$*), the solution* w', b' *on the labeled examples given by* (S_+, S'_-) *is identical to the solution* w, b *on the labeled examples* (S_+, S_-).

We remark that SVM satisfies a stronger notion of IIA, where the positives can also be a subset $S'_+ \subseteq S_+$ such that the set of consistent hypothesis remains the same. However the above version suffices for our purposes.

Proof. Consider the SVM solution w, b (for (14)) on labeled data S_+, S_-. This solution is unique: if there are two minimizing solutions $(w, b), (w', b')$ then the solution $(\frac{1}{2}(w+w'), \frac{1}{2}(b+b'))$ also satisfies all the constraints of (14) but attains a smaller objective value due to strong convexity.

Consider the classifier w', b' that attains the optimum margin for the dataset S_+, S'_-. From Corollary 2.10 (and Theorem 2.7) classifier $(w', b') \in \mathcal{H}(S_+, S_-)$. Moreover the (minimum) margin over S_+, S_- is attained by a point in $S_+ \cup S'_-$ i.e., $\min_{x \in S_+ \cup S'_-} |w' \cdot x + b'| = \min_{x \in S_+ \cup S_-} |w' \cdot x + b'| =: \tau$.[3] Suppose not. There exists a point $x' \notin S_+ \cup S'_-$ with label y' (say $y' = -1$) such that $y'(w' \cdot x' + b') = \tau$, but for all labeled examples (x, y) given by $S_+ \cup S'_-$, $y(w' \cdot x + b') > \tau$. Then $x' \in S_+ \cup C^*(S_+) \subseteq S_+ \cup S'_-$ (if $y = -1$, $x' \in C^*(S_+)$) which gives a contradiction.

Hence w', b' also achieves the same margin on S_+, S_- (as it does on $S_+, C^*(S_+)$). This implies that w', b' is also a optimum margin classifier on S_+, S_- (since it had a larger margin than w, b on A^*_+, A^*_-). Since the solution to (14) is unique, we conclude that $(w, b) = (w', b')$. □

[3]Note that not all of the support vectors from Fact 3.5 need to be in $S_+ \cup S'_-$.

4 EXTENSIONS TO KERNEL CLASSIFIERS

Our results for linear classifiers in the previous section naturally extend to kernel-based classifiers like kernel support vector machines. Kernel-based classifiers can be much more expressive than linear classifiers as they embed the input space into a feature space that is high-dimensional (potentially infinite dimensional), where the data is potentially linearly separable. Popular kernels include the polynomial kernel, that can capture any polynomial threshold function (a classifier of the form $\mathrm{sign}(p(x))$ where $p(x)$ is any polynomial of x), radial basis kernels (e.g., Gaussian kernels) etc.

We start by recalling some notation and facts about kernels. In what follows, $\psi : \mathcal{X} \to \mathcal{V}$ embeds points in the input space $\mathcal{X}$ into a Hilbert space $\mathcal{V}$. The kernel function is given by the inner product $K(x, x') = \langle \psi(x), \psi(x') \rangle_{\mathcal{V}}$, and we assume that this can be computed in polynomial time given x, x'. Given a set of points $x_1, \ldots, x_m \in \mathcal{X}$, the $m \times m$ matrix formed with the (i, j)th entry $K(x_i, x_j)$ is positive semi-definite. The following standard theorem states that one can find the maximum margin classifier in the feature space $\psi(\mathcal{X})$ for a given set of m samples, by solving a simpler convex optimization problem over m dimensions.

Theorem 4.1 (see e.g., Chapter 16 of Shalev-Shwartz and Ben-David [11]). *For a given set of samples* $(x_1, y_1), \ldots, (x_m, y_m) \in \mathcal{X} \times \{\pm 1\}$*, consider the Hard-SVM problem (or the maximum margin classifier problem):*

$$\min_{w \in \mathcal{V}, b \in \mathbb{R}} \|w\|_2^2, \text{ s.t. } \forall i \in [m],\ y_i\Big(w \cdot \psi(x_i) + b\Big) \geqslant 1. \tag{15}$$

An optimal solution (w^*, b^*) *to* (15) *can be obtained in polynomial time by finding an optimal solution to the following convex optimization problem and setting* $w^* = \sum_{i=1}^m \alpha_i^* \psi(x_i)$*:*

$$(\alpha^*, b^*) = \operatorname*{arg\,min}_{\alpha \in \mathbb{R}^m, b \in \mathbb{R}} \sum_{i,j \in [m]} K(x_i, x_j)\alpha_i \alpha_j,$$
$$\text{s.t. } \forall i \in [m],\ y_i\Big(\sum_{j \in [m]} K(x_i, x_j)\alpha_j + b\Big) \geqslant 1. \tag{16}$$

Moreover the corresponding linear classifier is given by $h(x) = \mathit{sign}\big(\sum_{j \in [m]} \alpha_j K(x_j, x) + b\big)$.

Our main observation is that implementing the protocol in the previous section only involves solving a set of linear classification problems. These include the computation of the set $C^*(A_+)$ by Trent (one problem for each point in S_-), and the computation of the set $\mathrm{Leak}(A_+, A_-)$ by Trent (one problem for each point in S). These linear classification problems are solved using the Hard-SVM problem (maximum margin linear classifier), which also satisfies the IIA property. We can carry out the same arguments as in the previous sections on the points $\{(\psi(x_i), y_i) : i \in [m]\}$ by only accessing inner products $\{\psi(x_i) \cdot \psi(x_j) : i, j \in [m]\}$ (through the kernel function). Hence from Theorem 4.1 we immediately obtain the generalization of our guarantees to kernel-based classifiers.

Theorem 4.2. *Suppose a kernel function* $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is specified as above and efficiently computable (and known to Alice, Bob and Trent). Given a set of points* $S = (S_+, S_-)$ *that is realizable (w.r.t. to linear classifiers over the space* $\psi(\mathcal{X})$*), there is a protocol (the critical points protocol implemented using kernel SVM) that is*

correct, minimal, computationally efficient and truthful. Moreover, if the data set U is realizable (w.r.t. linear classifiers over the space $\psi(\mathcal{X})$), Bob uses the kernel SVM algorithm in Step 6 of Algorithm 3 (MPeD), then the outputs of Algorithm 2 (SPeD) and Algorithm 3 (MPeD) are identical assuming the same sampling procedure (for generating S from U).

5 DISCUSSION AND OPEN QUESTIONS

In this paper, we considered a multi-party classification problem that is motivated by e-discovery. We designed a protocol (critical points protocol) in the realizable setting of linear classifiers and kernel SVMs, that is correct, minimal, computationally efficient and truthful. Moreover this protocol fits into a machine learning framework along with any classifier that satisfies the a natural property called IIA; we provide a reduction to the standard single-party setting, thereby demonstrating that there is no loss in statistical efficiency.

The most natural direction for future research is to generalize to the non-realizable setting, where there is no perfect classifier from the hypothesis class. Here we may need to relax our requirement of computational efficiency, as the problem of learning a linear classifier in the non-realizable setting (also called agnostic learning) is known to be computationally intractable even in the single-party setting (Shalev-Shwartz and Ben-David [11]). However one may be able to obtain efficient protocols assuming access to an efficient (single-party) learning algorithm.

REFERENCES

[1] D. Bertsimas and J.N. Tsitsiklis. 1997. *Introduction to linear optimization.* Athena Scientific.

[2] Gordon V Cormack and Maura R Grossman. 2014. Evaluation of machine-learning protocols for technology-assisted review in electronic discovery. In *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval.* 153–162.

[3] Cynthia Dwork, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Richard Zemel. 2012. Fairness through awareness. In *Proceedings of the 3rd innovations in theoretical computer science conference.* 214–226.

[4] Jonah B Gelbach and Bruce H Kobayashi. 2015. The Law and Economics of Proportionality in Discovery. *Ga. L. Rev.* 50 (2015), 1093.

[5] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to solve any protocol problem. In *Proc. of STOC.*

[6] Shafi Goldwasser, Guy N Rothblum, Jonathan Shafer, and Amir Yehudayoff. 2021. Interactive proofs for verifying machine learning. In *12th Innovations in Theoretical Computer Science Conference.* Schloss Dagstuhl-Leibniz-Zentrum für Informatik.

[7] Maura R Grossman and Gordon V Cormack. 2010. Technology-assisted review in e-discovery can be more effective and more efficient than exhaustive manual review. *Rich. JL & Tech.* 17 (2010), 1.

[8] Arthur Jacot, Franck Gabriel, and Clément Hongler. 2018. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada). Curran Associates Inc., Red Hook, NY, USA, 8580–8589.

[9] Bernhard Scholkopf and Alexander J. Smola. 2001. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond.* MIT Press, Cambridge, MA, USA.

[10] Alexander Schrijver. 1999. Theory of linear and integer programming. In *Wiley-Interscience series in discrete mathematics and optimization.*

[11] Shai Shalev-Shwartz and Shai Ben-David. 2014. *Understanding Machine Learning - From Theory to Algorithms.* Cambridge University Press. I–XVI, 1–397 pages.

[12] Andrew Chi-Chih Yao. 1986. How to generate and exchange secrets. In *27th Annual Symposium on Foundations of Computer Science.* IEEE, 162–167.